

also passed as a parameter, the *on()* method can have a switch statement to deal with messages based on the type.

The simplest possible implementation of the *Broker* can store all the registered subscribers in memory.

```
package com.glarimy.framework;

import java.util.HashMap;
import java.util.Map;

public class MessageBroker implements Broker {
    private Map<Long, Subscriber> subscribers;

    public MessageBroker() {
        subscribers = new HashMap<Long, Subscriber>();
    }
    ...
}
```

The *subscribe()* method generates a unique ID before storing the subscriber in its list. A rudimentary implementation can use the current time as the ID, though it is not the best option for obvious reasons.

```
@Override
public long subscribe(Subscriber subscriber) {
    long sid = new Date().getTime();
    subscribers.put(sid, subscriber);
    return sid;
}
```

The *unsubscribe()* method requires no explanation.

```
@Override
public void unsubscribe(long sid) {
    subscribers.remove(sid);
}
```

And the *publish()* method iterates through all the subscribers and calls the *on()* method on each one of them.

```
@Override
public void publish(String message, String type) {
    subscribers.values().stream().forEach(s -> s.on(message, type));
}
```

Note that this implementation does not give the non-blocking flavour of asynchronous communication. It only decouples the consumers and the providers. In order for real non-blocking communication, the *publish()* method can make use of a thread pool and call the *on()* method in separate threads.

This setup is depicted in Figure 1.

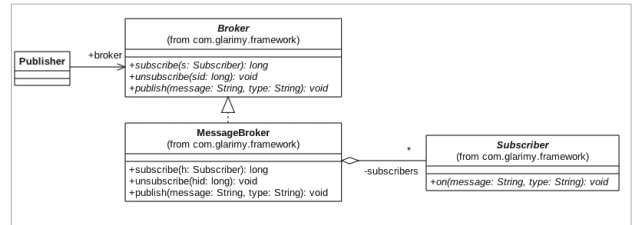


Figure 1: Messaging infrastructure

One semantic limitation of the above design is that a subscriber receives all the messages and uses a *switch* statement to handle them. This does not help us in improving the modularity of the application. When a new type of message is to be handled, the *on()* method of the subscriber needs to be modified with an additional case in the switch. Similar modifications are expected also when a type of message is to be ignored or handled differently.

Instead, an interface like *Handler* in the following snippet can be designed only to handle one type of message.

```
package com.glarimy.framework;

public interface Handler {
    public void handle(String message);
}
```

As can be observed, the handler just receives a message that can be handled without a switch statement. In other words, a given application can have different handler implementations for different types of messages.

Accordingly, the broker is refactored to register handlers, instead of subscribers.

```
package com.glarimy.framework;

public interface Broker {
    public long register(Handler handler);
    public void unregister(long hid);
    public void publish(String message, String type);
}
```

The broker implementation maintains a list of handlers and delivers the messages only to the matching handlers based on the type of message published. In order to identify the type of message that a handler is interested in, the interface can be updated as follows:

```
package com.glarimy.framework;

public interface Handler {
```